

UNIT- II

UNIT – II

ARM Programming - Instruction Set - Data Processing Instructions - Addressing Modes - Branch, Load, Store Instructions - PSR Instructions - Conditional Instructions.

ARM Programming

ARM Design Philosophy

- **ARM core uses RISC architecture**
 - Reduced instruction set
 - Load store architecture
 - Large number of general purpose registers
 - Parallel executions with pipelines

But some differences from RISC

- Enhanced instructions for
 - Thumb mode
 - DSP instructions
 - Conditional execution instruction
 - 32 bit barrel shifter

Consider an expression,

$$A = B + C$$

To evaluate the expression

- Load A to a general purpose register R1
- Load B to a general purpose register R2
- Load C to a general purpose register R3
- ADD R1, R2, R3 – adds the content of R2, R3 and stores it in R1
- Store R1 to A

ARM INSTRUCTION SET

Different ARM architecture revisions support different instructions. ISA includes all the core ARM instructions as well as some of the newer features in the ARM instruction set.

Mnemonics	ARM ISA	Description
ADC	v1	add two 32-bit values and carry
ADD	v1	add two 32-bit values
AND	v1	logical bitwise AND of two 32-bit values
B	v1	branch relative ± 32 MB
BIC	v1	logical bit clear (AND NOT) of two 32-bit values
BKPT	v5	breakpoint instructions
BL	v1	relative branch with link
BLX	v5	branch with link and exchange
BX	v4T	branch with exchange
CDP CDP2	v2 v5	coprocessor data processing operation
CLZ	v5	count leading zeros
CMN	v1	compare negative two 32-bit values
CMP	v1	compare two 32-bit values
EOR	v1	logical exclusive OR of two 32-bit values
LDC LDC2	v2 v5	load to coprocessor single or multiple 32-bit values
LDM	v1	load multiple 32-bit words from memory to ARM registers
LDR	v1 v4 v5E	load a single value from a virtual address in memory
MCR MCR2 MCRR	v2 v5 v5E	move to coprocessor from an ARM register or registers
MLA	v2	multiply and accumulate 32-bit values
MOV	v1	move a 32-bit value into a register
MRC MRC2 MRRC	v2 v5 v5E	move to ARM register or registers from a coprocessor
MRS	v3	move to ARM register from a status register (<i>cpsr</i> or <i>spsr</i>)
MSR	v3	move to a status register (<i>cpsr</i> or <i>spsr</i>) from an ARM register
MUL	v2	multiply two 32-bit values
MVN	v1	move the logical NOT of 32-bit value into a register
ORR	v1	logical bitwise OR of two 32-bit values
PLD	v5E	preload hint instruction
QADD	v5E	signed saturated 32-bit add
QDADD	v5E	signed saturated double and 32-bit add
QDSUB	v5E	signed saturated double and 32-bit subtract
QSUB	v5E	signed saturated 32-bit subtract
RSB	v1	reverse subtract of two 32-bit values
RSC	v1	reverse subtract with carry of two 32-bit integers
SBC	v1	subtract with carry of two 32-bit values
SMLAxy	v5E	signed multiply accumulate instructions ($((16 \times 16) + 32 = 32\text{-bit})$)
SMLAL	v3M	signed multiply accumulate long ($((32 \times 32) + 64 = 64\text{-bit})$)
SMLALxy	v5E	signed multiply accumulate long ($((16 \times 16) + 64 = 64\text{-bit})$)
SMLAWy	v5E	signed multiply accumulate instruction ($((32 \times 16) \gg 16) + 32 = 32\text{-bit})$)
SMULL	v3M	signed multiply long ($32 \times 32 = 64\text{-bit})$)

Mnemonics	ARM ISA	Description
SMULxy	v5E	signed multiply instructions ($16 \times 16 = 32$ -bit)
SMULWy	v5E	signed multiply instruction ($((32 \times 16) \gg 16 = 32$ -bit)
STC STC2	v2 v5	store to memory single or multiple 32-bit values from coprocessor
STM	v1	store multiple 32-bit registers to memory
STR	v1 v4 v5E	store register to a virtual address in memory
SUB	v1	subtract two 32-bit values
SWI	v1	software interrupt
SWP	v2a	swap a word/byte in memory with a register, without interruption
TEQ	v1	test for equality of two 32-bit values
TST	v1	test for bits in a 32-bit value
UMLAL	v3M	unsigned multiply accumulate long ($((32 \times 32) + 64 = 64$ -bit)
UMULL	v3M	unsigned multiply long ($32 \times 32 = 64$ -bit)

- ARM instructions process data held in registers and only access memory with load and store instructions.
- ARM instructions commonly take two or three operands.

For instance the ADD instruction below adds the two values stored in registers *r1* and *r2* (the source registers). It writes the result to register *r3* (the destination register).

Instruction Syntax	Destination register (<i>Rd</i>)	Source register 1 (<i>Rn</i>)	Source register 2 (<i>Rm</i>)
ADD <i>r3</i> , <i>r1</i> , <i>r2</i>	<i>r3</i>	<i>r1</i>	<i>r2</i>

DATA PROCESSING INSTRUCTIONS

The data processing instructions manipulate data within registers. They are move instructions, arithmetic instructions, logical instructions, comparison instructions, and multiply instructions. Most data processing instructions can process one of their operands using the barrel shifter.

- If you use the S suffix on a data processing instruction, then it updates the flags in the cpsr.
- Move and logical operations update the carry flag *C*, negative flag *N*, and zero flag *Z*.
- The carry flag is set from the result of the barrel shift as the last bit shifted out.
- The *N* flag is set to bit 31 of the result. The *Z* flag is set if the result is zero.

MOVE INSTRUCTIONS

- Move is the simplest ARM instruction.
- It copies *N* into a destination register *Rd*, where *N* is a register or immediate value.
- This instruction is useful for setting initial values and transferring data between registers.

Syntax: <instruction>{<cond>}{S} *Rd*, *N*

MOV	Move a 32-bit value into a register	$Rd = N$
MVN	move the NOT of the 32-bit value into a register	$Rd = \sim N$

MOV Move a 32-bit value into a register

MOV<cond>{S} Rd, #<rotated_immed>

Rd = <rotated_immed>

CPSR Updated if S suffix specified

Example

This example shows a simple move instruction. The MOV instruction takes the contents of register *r5* and copies them into register *r7*, in this case, taking the value 5, and overwriting the value 8 in register *r7*.

PRE

r5 = 5

r7 = 8

MOV *r7*, *r5*; let *r7* = *r5*

POST

r5 = 5

r7 = 5

BARREL SHIFTER

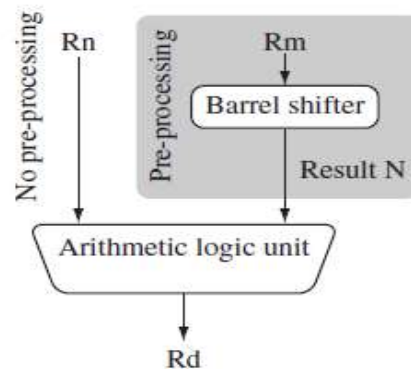


Fig. Barrel shifter and ALU

- *N* can be more than just a register or immediate value; it can also be a register *Rm* that has been preprocessed by the barrel shifter prior to being used by a data processing instruction.
- Data processing instructions are processed within the arithmetic logic unit (ALU).
- A unique and powerful feature of the ARM processor is the ability to shift the 32-bit binary pattern in one of the source registers left or right by a specific number of positions before it enters the ALU.
- This shift increases the power and flexibility of many data processing operations.
- There are data processing instructions that do not use the barrel shift, for example, the MUL (multiply), CLZ (count leading zeros), and QADD (signed saturated 32-bit add) instructions.
- Pre-processing or shift occurs within the cycle time of the instruction.
- This is particularly useful for loading constants into a register and achieving fast multiplies or division by a power of 2.

MOV<cond>{S} Rd, Rm {, <shift>}

Rd = <shifted_Rm>

CPSR Updated if S suffix specified

Example

Apply a logical shift left (LSL) to register *Rm* before moving it to the destination register. This is the same as applying the standard C language shift operator `<` to the register. The MOV instruction copies the shift operator result *N* into register *Rd*. *N* represents the result of the LSL operation

PRE

r5 = 5

r7 = 8

MOV r7, r5, LSL #2 ; let r7 = r5*4 = (r5 << 2)

POST

r5 = 5

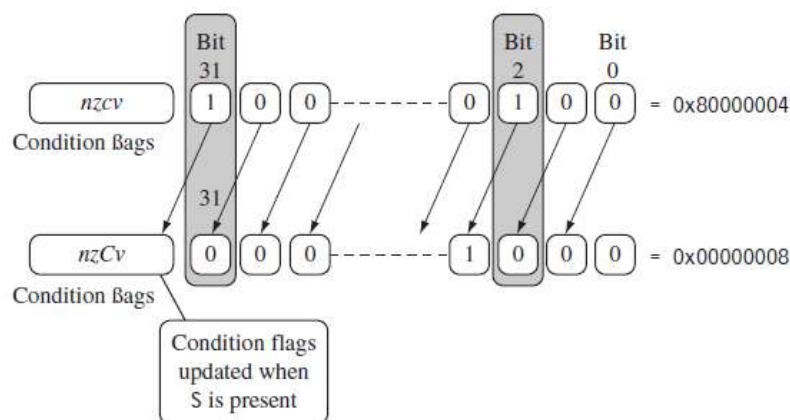
r7 = 20

The example multiplies register *r5* by four and then places the result into register *r7*.

Logical Instructions:

Mnemonic	Description	Shift	Result	Shift amount <i>y</i>
LSL	logical shift left	$x\text{LSL } y$	$x \ll y$	#0–31 or <i>Rs</i>
LSR	logical shift right	$x\text{LSR } y$	$(\text{unsigned})x \gg y$	#1–32 or <i>Rs</i>
ASR	arithmetic right shift	$x\text{ASR } y$	$(\text{signed})x \gg y$	#1–32 or <i>Rs</i>
ROR	rotate right	$x\text{ROR } y$	$((\text{unsigned})x \gg y) (x \ll (32 - y))$	#1–31 or <i>Rs</i>
RRX	rotate right extended	$x\text{RRX}$	$(c\text{ flag} \ll 31) ((\text{unsigned})x \gg 1)$	none

Note: *x* represents the register being shifted and *y* represents the shift amount.



Logical shift left by one.

Barrel shift operation syntax for data processing instructions.

N shift operations	Syntax
Immediate	#immediate
Register	Rm
Logical shift left by immediate	Rm, LSL #shift_imm
Logical shift left by register	Rm, LSL Rs
Logical shift right by immediate	Rm, LSR #shift_imm
Logical shift right with register	Rm, LSR Rs
Arithmetic shift right by immediate	Rm, ASR #shift_imm
Arithmetic shift right by register	Rm, ASR Rs
Rotate right by immediate	Rm, ROR #shift_imm
Rotate right by register	Rm, ROR Rs
Rotate right with extend	Rm, RRX

MVN

Move the logical not of a 32-bit value into a register

MVN<cond>{S} Rd, #<rotated_immed>

$Rd = \sim \text{<rotated_immed>}$

CPSR Updated if S suffix specified

ARITHMETIC INSTRUCTIONS

The arithmetic instructions implement addition and subtraction of 32-bit signed and unsigned values.

Syntax: <instruction>{<cond>}{S} Rd, Rn, N

ADC	add two 32-bit values and carry	$Rd = Rn + N + \text{carry}$
ADD	add two 32-bit values	$Rd = Rn + N$
RSB	reverse subtract of two 32-bit values	$Rd = N - Rn$
RSC	reverse subtract with carry of two 32-bit values	$Rd = N - Rn - !(\text{carry flag})$
SBC	subtract with carry of two 32-bit values	$Rd = Rn - N - !(\text{carry flag})$
SUB	subtract two 32-bit values	$Rd = Rn - N$

N is the result of the shifter operation.

ADD

Add two 32-bit values

ADD<cond>{S} Rd, Rn, #<rotated_immed>

$Rd = Rn + \langle \text{rotated_immed} \rangle$

CPSR Updated if S suffix specified

Eg:

(i) ADD r0, r1, #4 ; $r0 = r1 + 4$

(ii) ADDS r0, r2, r2 ; $r0 = r2 + r2$ and flags updated

ADC

Add two 32-bit values and carry

ADC<cond>{S} Rd, Rn, #<rotated_immed>

$Rd = Rn + \langle \text{rotated_immed} \rangle + C$ CPSR Updated if S suffix specified

Eg:

ADC r1, r1, r3

SUB

Subtract instruction subtracts a value stored in register *r2* from a value stored in register *r1*. The result is stored in register *r0*.

Eg:

PRE

$r0 = 0x00000000$

$r1 = 0x00000002$

$r2 = 0x00000001$

SUB r0, r1, r2

POST

$r0 = 0x00000001$

RSB

Reverse subtract instruction (RSB) subtracts *r1* from the constant value #0, writing the result to *r0*. You can use this instruction to negate numbers.

Eg:

PRE

$r0 = 0x00000000$

$r1 = 0x00000077$

RSB r0, r1, #0 ; $Rd = 0x0 - r1$

POST

$r0 = -r1 = 0xfffff89$

SUBS

The SUBS instruction is useful for decrementing loop counters. In this example we subtract the immediate value one from the value one stored in register *r1*. The result value zero is written to register *r1*. The *cpsr* is updated with the ZC flags being set.

Eg:

PRE

cpsr = nzcqvqiFt_USER
r1 = 0x00000001
SUBS r1, r1, #1

POST

cpsr = nZCvqiFt_USER
r1 = 0x00000000

SBC

Subtract with carry

SBC<cond>{S} Rd, Rn, #<rotated_immed>

Rd = Rn - <rotated_immed> - (~C) Updated if S suffix specified

Eg:

SBC r1, r1, r3 ; subtract high words and borrow

USING THE BARREL SHIFTER WITH ARITHMETIC INSTRUCTIONS

Example

Register *r1* is first shifted one location to the left to give the value of twice *r1*. The ADD instruction then adds the result of the barrel shift operation to register *r1*. The final result transferred into register *r0* is equal to three times the value stored in register *r1*.

PRE

r0 = 0x00000000
r1 = 0x00000005
ADD r0, r1, r1, LSL #1

POST

r0 = **0x0000000f**
r1 = 0x00000005

LOGICAL INSTRUCTIONS

Logical instructions perform bitwise logical operations on the two source registers.

Syntax: <instruction>{<cond>}{S} Rd, Rn, N

AND	logical bitwise AND of two 32-bit values	$Rd = Rn \& N$
ORR	logical bitwise OR of two 32-bit values	$Rd = Rn N$
EOR	logical exclusive OR of two 32-bit values	$Rd = Rn \wedge N$
BIC	logical bit clear (AND NOT)	$Rd = Rn \& \sim N$

AND

Logical bitwise AND of two 32-bit values

(i) AND<cond>{S} Rd, Rn, #<rotated_immed>

$Rd = Rn \& \text{<rotated_immed>}$

CPSR Updated if S suffix specified

(ii) $\text{AND<cond>\{S\} } Rd, Rn, Rm \{, \text{<shift>\}$

$Rd = Rn \& \text{<shifted_Rm>}$

CPSR Updated if S suffix specified

Examples

AND r0, r0, #0xFF

- $r0 = r0 \& 0xFF$

ANDS r0, r0, r1

- $r0 = r0 \& r1$
- CPSR updated

ORR

Logical OR operation between registers *r1* and *r2*. *r0* holds the result.

Example:

PRE

$r0 = 0x00000000$

$r1 = 0x02040608$

$r2 = 0x10305070$

ORR r0, r1, r2

POST

$r0 = 0x12345678$ ■

EOR

Logical exclusive OR of two 32-bit values

(i) $\text{EOR<cond>\{S\} } Rd, Rn, \# \text{<rotated_immed>}$

$Rd = Rn \wedge \text{<rotated_immed>}$ Updated if S suffix specified

(ii) $\text{EOR<cond>\{S\} } Rd, Rn, Rm \{, \text{<shift>\}$

$Rd = Rn \wedge \text{<shifted_Rm>}$ Updated if S suffix specified

BIC

Logical bit clear (AND NOT) of two 32-bit values

(i) $\text{BIC<cond>\{S\} } Rd, Rn, \# \text{<rotated_immed>}$

$Rd = Rn \& \sim \text{<rotated_immed>}$ Updated if S suffix specified

Example

PRE

r1 = 0b1111
 r2 = 0b0101
 BIC r0, r1, r2

POST

r0 = **0b1010**

This is equivalent to $Rd = Rn \text{ AND NOT}(N)$

Register *r2* contains a binary pattern where every binary 1 in *r2* clears a corresponding bit location in register *r1*. This instruction is particularly useful when clearing status bits and is frequently used to change interrupt masks in the *cpsr*.

COMPARISON INSTRUCTIONS

The comparison instructions are used to compare or test a register with a 32-bit value. They update the *cpsr* flag bits according to the result, but do not affect other registers. After the bits have been set, the information can then be used to change program flow by using conditional execution.

Syntax: <instruction>{<cond>} Rn, N

CMN	compare negated	flags set as a result of $Rn + N$
CMP	compare	flags set as a result of $Rn - N$
TEQ	test for equality of two 32-bit values	flags set as a result of $Rn \wedge N$
TST	test bits of a 32-bit value	flags set as a result of $Rn \& N$

CMP

Compare two 32-bit integers

(i) CMP<cond> Rn, #<rotated_immed>

cpsr flags set on the result of $(Rn - \text{<rotated_immed>})$

(ii) CMP<cond> Rn, Rm {, <shift>}

cpsr flags set on the result of $(Rn - \text{<shifted_Rm>})$

Example

The value of the *z* flag prior to execution is 0 and is represented by a lowercase *z*. After execution the *z* flag changes to 1 or an uppercase *Z*. This change indicates *equality*.

PRE

cpsr = nzcqvqiFt_USER
 r0 = 4
 r9 = 4
 CMP r0, r9

POST

cpsr = nZcqvqiFt_USER

- The CMP is effectively a subtract instruction with the result discarded

CMN

Compare negative

(i) CMN<cond> Rn, #<rotated_immed>

cpsr flags set on the result of (Rn + <rotated_immed>)

(ii) CMN<cond> Rn, Rm {, <shift>}

cpsr flags set on the result of (Rn + <shifted_Rm>)

TST & TEQ

- The TST instruction is a logical AND operation, and TEQ is a logical exclusive OR operation.
- For each, the results are discarded but the condition bits are updated in the *cpsr*.

TST

Test bits of a 32-bit value

TST<cond> Rn, #<rotated_immed>

Example

TST r0, #0xFF ; test if the bottom 8 bits of r0 are 0

TEQ

Test for equality of two 32-bit values

TEQ<cond> Rn, #<rotated_immed>

Example

TEQ r0, #1 ; test to see if r0==1

MULTIPLY INSTRUCTIONS

- The multiply instructions multiply the contents of a pair of registers and, depending upon the instruction, accumulate the results in with another register.
- The long multiplies accumulate onto a pair of registers representing a 64-bit value.
- The final result is placed in a destination register or a pair of registers.

Syntax: MLA{<cond>}{S} Rd, Rm, Rs, Rn
 MUL{<cond>}{S} Rd, Rm, Rs

MLA	multiply and accumulate	$Rd = (Rm * Rs) + Rn$
MUL	multiply	$Rd = Rm * Rs$

- The number of cycles taken to execute a multiply instruction depends on the processor implementation.

- For some implementations the cycle timing also depends on the value in *Rs*.

Example

PRE

r0 = 0x00000000
r1 = 0x00000002
r2 = 0x00000002

MUL r0, r1, r2 ; r0 = r1*r2

POST

r0 = 0x00000004
r1 = 0x00000002
r2 = 0x00000002

Long Multiply Instructions:

Syntax: <instruction>{<cond>}{S} RdLo, RdHi, Rm, Rs

SMLAL	signed multiply accumulate long	$[RdHi, RdLo] = [RdHi, RdLo] + (Rm * Rs)$
SMULL	signed multiply long	$[RdHi, RdLo] = Rm * Rs$
UMLAL	unsigned multiply accumulate long	$[RdHi, RdLo] = [RdHi, RdLo] + (Rm * Rs)$
UMULL	unsigned multiply long	$[RdHi, RdLo] = Rm * Rs$

- The long multiply instructions (SMLAL, SMULL, UMLAL, and UMULL) produce a 64-bit result.
- The result is too large to fit a single 32-bit register so the result is placed in two registers labeled *RdLo* and *RdHi*.
 - RdLo* holds the lower 32 bits of the 64-bit result
 - RdHi* holds the higher 32 bits of the 64-bit result.

Example

PRE

r0 = 0x00000000
r1 = 0x00000000
r2 = 0xf0000002
r3 = 0x00000002

UMULL r0, r1, r2, r3 ; [r1,r0] = r2*r3

POST

r0 = 0xe0000004 ; = RdLo
r1 = 0x00000001 ; = RdHi

BRANCH INSTRUCTIONS

- A branch instruction changes the flow of execution or is used to call a routine.
- This type of instruction allows programs to have subroutines, *if-then-else* structures, and loops.
- The change of execution flow forces the program counter *pc* to point to a new address.
- The ARMv5E instruction set includes four different branch instructions.

Syntax: B{<cond>} label
 BL{<cond>} label
 BX{<cond>} Rm
 BLX{<cond>} label | Rm

B	branch	$pc = label$
BL	branch with link	$pc = label$ $lr = \text{address of the next instruction after the BL}$
BX	branch exchange	$pc = Rm \ \& \ 0xfffffffffe, T = Rm \ \& \ 1$
BLX	branch exchange with link	$pc = label, T = 1$ $pc = Rm \ \& \ 0xfffffffffe, T = Rm \ \& \ 1$ $lr = \text{address of the next instruction after the BLX}$

- The address *label* is stored in the instruction as a signed *pc*-relative offset and must be within approximately 32 MB of the branch instruction.
- *T* refers to the Thumb bit in the *cpsr*.
- When instructions set *T*, the ARM switches to Thumb state.

Example: 1

The forward branch skips three instructions. The backward branch creates an infinite loop.

B forward

ADD r1, r2, #4

ADD r0, r6, #2

ADD r3, r7, #4

forward

SUB r1, r2, #4

backward

ADD r1, r2, #4

SUB r1, r2, #4

ADD r4, r6, r7

B backward

- Branches are used to change execution flow.
- Most assemblers hide the details of a branch instruction encoding by using labels.
- In this example, *forward* and *backward* are the labels.
- The branch labels are placed at the beginning of the line and are used to mark an address that can be used later by the assembler to calculate the branch offset.

Example: 2

The branch with link, or BL, instruction is similar to the B instruction but overwrites the link register *lr* with a return address. It performs a subroutine call. This example shows a simple fragment of code that branches to a subroutine using the BL instruction. To return from a subroutine, you copy the link register to the *pc*.

```
BL subroutine      ; branch to subroutine
CMP r1, #5         ; compare r1 with 5
MOVEQ r1, #0       ; if (r1==5) then r1 = 0
:
subroutine
<subroutine code>
MOV pc, lr         ; return by moving pc = lr
```

BX and BLX

The branch exchange (BX) and branch exchange with link (BLX) are the third type of branch instruction. The BX instruction uses an absolute address stored in register *Rm*. It is primarily used to branch to and from Thumb code. The *T* bit in the *cpsr* is updated by the least significant bit of the branch register. Similarly the BLX instruction updates the *T* bit of the *cpsr* with the least significant bit and additionally sets the link register with the return address.

BX

Branch with exchange (branch with possible state switch)

BX<cond> *Rm*

$pc = Rm \& 0xffffffe$ $T = Rm \& 1$

BLX

BLX Branch with link and exchange (subroutine call with possible state switch)

BLX <address25>

$lr = ret+0$; $pc = \text{<address25>}$ $T=1$ (switch to Thumb state)

BLX<cond> *Rm*

$lr = ret+0$; $pc = Rm \& 0xffffffe$ $T = Rm \& 1$

Example

BLX thumb_code ; call a Thumb subroutine from ARM state

BLX r0 ; call the subroutine pointed to by r0;

ARM code if r0 even, Thumb if r0 odd

LOAD-STORE INSTRUCTIONS

Load-store instructions transfer data between memory and processor registers. There are three types of load-store instructions: single-register transfer, multiple-register transfer, and swap.

Single-Register Transfer

These instructions are used for moving a single data item in and out of a register. The datatypes supported are signed and unsigned words (32-bit), halfwords (16-bit), and bytes.

Syntax: <LDR|STR>{<cond>}{B} Rd, addressing¹
LDR{<cond>}SB|H|SH Rd, addressing²
STR{<cond>}H Rd, addressing²

LDR	load word into a register	$Rd \leftarrow mem32[address]$
STR	save byte or word from a register	$Rd \rightarrow mem32[address]$
LDRB	load byte into a register	$Rd \leftarrow mem8[address]$
STRB	save byte from a register	$Rd \rightarrow mem8[address]$
LDRH	load halfword into a register	$Rd \leftarrow mem16[address]$
STRH	save halfword into a register	$Rd \rightarrow mem16[address]$
LDRSB	load signed byte into a register	$Rd \leftarrow SignExtend(mem8[address])$
LDRSH	load signed halfword into a register	$Rd \leftarrow SignExtend(mem16[address])$

LDR data types

Load	Datatype	<size> (bytes)	Action
LDR	word	4	$Rd = memory(a, 4)$
LDRB	unsigned Byte	1	$Rd = (zero\text{-}extend)memory(a, 1)$
LDRBT	Byte Translated	1	$Rd = (zero\text{-}extend)memoryT(a, 1)$
LDRD	Double word	8	$Rd = memory(a, 4)$ $R(d+1) = memory(a+4, 4)$
LDREX	word EXclusive	4	$Rd = memoryEx(a, 4)$
LDRH	unsigned Halfword	2	$Rd = (zero\text{-}extend)memory(a, 2)$
LDRSB	Signed Byte	1	$Rd = (sign\text{-}extend)memory(a, 1)$
LDRSH	Signed Halfword	2	$Rd = (sign\text{-}extend)memory(a, 2)$
LDRT	word Translated	4	$Rd = memoryT(a, 4)$

Example

LDR and STR instructions can load and store data on a boundary alignment that is the same as the data type size being loaded or stored. For example, LDR can only load 32-bit words on a memory address that is a multiple of four bytes—0, 4, 8, and so on.

Load register r0 with the contents of the memory address pointed to by register r1

LDR r0, [r1] ; = LDR r0, [r1, #0]

Store the contents of register r0 to the memory address pointed to by register r1.

STR r0, [r1] ; = STR r0, [r1, #0]

- The first instruction loads a word from the address stored in register *r1* and places it into register *r0*.
- The second instruction goes the other way by storing the contents of register *r0* to the address contained in register *r1*.
- The offset from register *r1* is zero.
- Register *r1* is called the *base address register*.

Single-Register Load-Store Addressing Modes

Index modes:

The ARM instruction set provides different modes for addressing memory. These modes incorporate one of the indexing methods: preindex with writeback, preindex, and postindex.

Index methods.

Index method	Data	Base address register	Example
Preindex with writeback	<i>mem[base + offset]</i>	<i>base + offset</i>	LDR r0, [r1, #4] !
Preindex	<i>mem[base + offset]</i>	<i>not updated</i>	LDR r0, [r1, #4]
Postindex	<i>mem[base]</i>	<i>base + offset</i>	LDR r0, [r1], #4

Note: ! indicates that the instruction writes the calculated address back to the base address register.

- Preindex with writeback calculates an address from a base register plus address offset and then updates that address base register with the new address.
- In contrast, the preindex offset is the same as the preindex with writeback but does not update the address base register.
- Postindex only updates the address base register after the address is used.
- The preindex mode is useful for accessing an element in a data structure.
- The postindex and preindex with writeback modes are useful for traversing an array.

Example

PRE

r0 = 0x00000000

r1 = 0x00090000

mem32[0x00009000] = 0x01010101

mem32[0x00009004] = 0x02020202

LDR r0, [r1, #4]!

Preindexing with writeback:

POST(1)

```
r0 = 0x02020202  
r1 = 0x00009004  
LDR r0, [r1, #4]
```

Preindexing:

POST(2)

```
r0 = 0x02020202  
r1 = 0x00009000  
LDR r0, [r1], #4
```

Postindexing:

POST(3)

```
r0 = 0x01010101  
r1 = 0x00009004
```

Addressing modes:

Single-register load-store addressing, word or unsigned byte.

Addressing ¹ mode and index method	Addressing ¹ syntax
Preindex with immediate offset	[Rn, #+/-offset_12]
Preindex with register offset	[Rn, +/-Rm]
Preindex with scaled register offset	[Rn, +/-Rm, shift #shift_imm]
Preindex writeback with immediate offset	[Rn, #+/-offset_12]!
Preindex writeback with register offset	[Rn, +/-Rm]!
Preindex writeback with scaled register offset	[Rn, +/-Rm, shift #shift_imm]!
Immediate postindexed	[Rn], #+/-offset_12
Register postindex	[Rn], +/-Rm
Scaled register postindex	[Rn], +/-Rm, shift #shift_imm

Single-register load-store addressing, halfword, signed halfword, signed byte, and doubleword.

Addressing ² mode and index method	Addressing ² syntax
Preindex immediate offset	$[Rn, \#+/-offset_8]$
Preindex register offset	$[Rn, +/-Rm]$
Preindex writeback immediate offset	$[Rn, \#+/-offset_8]!$
Preindex writeback register offset	$[Rn, +/-Rm]!$
Immediate postindexed	$[Rn], \#+/-offset_8$
Register postindexed	$[Rn], +/-Rm$

- A signed offset or register is denoted by “+/-”, identifying that it is either a positive or negative offset from the base address register Rn .
- The base address register is a pointer to a byte in memory, and the offset specifies a number of bytes.
- *Immediate* means the address is calculated using the base address register and a 12-bit offset encoded in the instruction.
- *Register* means the address is calculated using the base address register and a specific register’s contents.
- *Scaled* means the address is calculated using the base address register and a barrel shift operation.

Multiple-Register Transfer

- Load-store multiple instructions can transfer multiple registers between memory and the processor in a single instruction.
- The transfer occurs from a base address register Rn pointing into memory.
- Multiple-register transfer instructions are more efficient from single-register transfers for moving blocks of data around memory and saving and restoring context and stacks.
- Load-store multiple instructions can increase interrupt latency.

Syntax: <LDM|STM>{<cond>}<addressing mode> $Rn\{!\}$,<registers>{^}

LDM	load multiple registers	$\{Rd\}^N \leftarrow mem32[start\ address + 4*N]$ optional Rn updated
STM	save multiple registers	$\{Rd\}^N \rightarrow mem32[start\ address + 4*N]$ optional Rn updated

- The base register Rn determines the source or destination address for a load store multiple instruction.
- This register can be optionally updated following the transfer.
- This occurs when register Rn is followed by the ! character

Addressing modes:

Addressing mode for load-store multiple instructions.

Addressing mode	Description	Start address	End address	$Rn!$
IA	increment after	Rn	$Rn + 4 * N - 4$	$Rn + 4 * N$
IB	increment before	$Rn + 4$	$Rn + 4 * N$	$Rn + 4 * N$
DA	decrement after	$Rn - 4 * N + 4$	Rn	$Rn - 4 * N$
DB	decrement before	$Rn - 4 * N$	$Rn - 4$	$Rn - 4 * N$

Example: 1

Register $r0$ is the base register Rn and is followed by $!$, indicating that the register is updated after the instruction is executed the “-” character is used to identify a range of registers. In this case the range is from register $r1$ to $r3$ inclusive. Each register can also be listed, using a comma to separate each register within “{” and “}” brackets.

PRE

```
mem32[0x80018] = 0x03
mem32[0x80014] = 0x02
mem32[0x80010] = 0x01
r0 = 0x00080010
r1 = 0x00000000
r2 = 0x00000000
r3 = 0x00000000
```

LDMIA $r0!$, { $r1$ - $r3$ }

POST

```
r0 = 0x0008001c
r1 = 0x00000001
r2 = 0x00000002
r3 = 0x00000003
```

Address pointer	Memory		
	address	Data	
$r0 = 0x80010 \rightarrow$	0x80020	0x00000005	$r3 = 0x00000000$ $r2 = 0x00000000$ $r1 = 0x00000000$
	0x8001c	0x00000004	
	0x80018	0x00000003	
	0x80014	0x00000002	
	0x80010	0x00000001	
	0x8000c	0x00000000	

Pre-condition for LDMIA instruction.

Address pointer	Memory		
	address	Data	
$r0 = 0x8001c \rightarrow$	0x80020	0x00000005	$r3 = 0x00000003$ $r2 = 0x00000002$ $r1 = 0x00000001$
	0x8001c	0x00000004	
	0x80018	0x00000003	
	0x80014	0x00000002	
	0x80010	0x00000001	
	0x8000c	0x00000000	

Post-condition for LDMIA instruction.

Example: 2

STM *increment before* instruction followed by an LDM *decrement after* instruction.

PRE

r0 = 0x00009000

r1 = 0x00000009

r2 = 0x00000008

r3 = 0x00000007

STMIB r0!, {r1-r3}

MOV r1, #1

MOV r2, #2

MOV r3, #3

PRE(2)

r0 = 0x0000900c

r1 = 0x00000001

r2 = 0x00000002

r3 = 0x00000003

LDMDA r0!, {r1-r3}

POST

r0 = 0x00009000

r1 = 0x00000009

r2 = 0x00000008

r3 = 0x00000007

Example: 3

This example is a simple routine that copies blocks of 32 bytes from a source address location to a destination address location. The example has two load-store multiple instructions, which use the same *increment after* addressing mode.

; r9 points to start of source data

; r10 points to start of destination data

; r11 points to end of the source loop

; load 32 bytes from source and update r9 pointer

LDMIA r9!, {r0-r7}

; store 32 bytes to destination and update r10 pointer

STMIA r10!, {r0-r7}

CMP r9, r11

BNE loop

STACK OPERATIONS

- The ARM architecture uses the load-store multiple instructions to carry out stack operations.
- The *pop* operation (removing data from a stack) uses a load multiple instruction.
- The *push* operation (placing data onto the stack) uses a store multiple instruction.
- A stack is either *ascending* (A) or *descending* (D).
 - Ascending stacks grow towards higher memory addresses.
 - Descending stacks grow towards lower memory addresses.
- In a *full stack* (F), the stack pointer *sp* points to an address that is the last used or full location (i.e., *sp* points to the last item on the stack).
- In an *empty stack* (E) the *sp* points to an address that is the first unused or empty location (i.e., it points after the last item on the stack)

Addressing modes:

Addressing methods for stack operations.

Addressing mode	Description	Pop	= LDM	Push	= STM
FA	full ascending	LDMFA	LDMDA	STMFA	STMIB
FD	full descending	LDMFD	LDMIA	STMFD	STMDB
EA	empty ascending	LDMEA	LDMDB	STMEA	STMIA
ED	empty descending	LDMED	LDMIB	STMED	STMDA

Example: 1

The STMFD instruction pushes registers onto the stack, updating the *sp*.

PRE

r1 = 0x00000002

r4 = 0x00000003

sp = 0x00080014

STMFD sp!, {r1,r4}

PRE	Address	Data	POST	Address	Data
	0x80018	0x00000001		0x80018	0x00000001
<i>sp</i> →	0x80014	0x00000002		0x80014	0x00000002
	0x80010	Empty		0x80010	0x00000003
	0x8000c	Empty	<i>sp</i> →	0x8000c	0x00000002

STMFD instruction—full stack push operation.

POST

r1 = 0x00000002

r4 = 0x00000003

sp = 0x0008000c

Example: 2

The STMED instruction pushes the registers onto the stack but updates register *sp* to point to the next empty location.

PRE

r1 = 0x00000002

r4 = 0x00000003

sp = 0x00080010

STMED *sp*!, {*r1*,*r4*}

POST *r1* = 0x00000002

r4 = 0x00000003

sp = 0x00080008

PRE	Address	Data	POST	Address	Data
	0x80018	0x00000001		0x80018	0x00000001
	0x80014	0x00000002		0x80014	0x00000002
<i>sp</i> →	0x80010	Empty		0x80010	0x00000003
	0x8000c	Empty		0x8000c	0x00000002
	0x80008	Empty	<i>sp</i> →	0x80008	Empty

STMED instruction—empty stack push operation.

- There are three attributes that need to be preserved:
 - the *stack base*
 - the *stack pointer* and
 - the *stack limit*.
- The stack base is the starting address of the stack in memory.
- The stack pointer initially points to the stack base; as data is pushed onto the stack, the stack pointer descends memory and continuously points to the top of stack.
- If the stack pointer passes the stack limit, then a *stack overflow error* has occurred.
- If the stack pointer goes back past the stack limit, then a *stack underflow error* has occurred.

SWAP INSTRUCTION

- The swap instruction is a special case of a load-store instruction.
- It swaps the contents of memory with the contents of a register.
- This instruction is an *atomic operation*—it reads and writes a location in the same bus operation, preventing any other instruction from reading or writing to that location until it completes.

Syntax: SWP{B} {<cond>} *Rd*,*Rm*, [*Rn*]

SWP	swap a word between memory and a register	$tmp = mem32[Rn]$ $mem32[Rn] = Rm$ $Rd = tmp$
SWPB	swap a byte between memory and a register	$tmp = mem8[Rn]$ $mem8[Rn] = Rm$ $Rd = tmp$

- Swap cannot be interrupted by any other instruction or any other bus access.
- The system “holds the bus” until the transaction is complete.

Example

The swap instruction loads a word from memory into register *r0* and overwrites the memory with register *r1*.

PRE

```
mem32[0x9000] = 0x12345678
r0 = 0x00000000
r1 = 0x11112222
r2 = 0x00009000
```

SWP *r0*, *r1*, [*r2*]

POST

```
mem32[0x9000] = 0x11112222
r0 = 0x12345678
r1 = 0x11112222
r2 = 0x00009000
```

This instruction is particularly useful when implementing semaphores and mutual exclusion in an operating system.

SOFTWARE INTERRUPT INSTRUCTION

A software interrupt instruction (SWI) causes a software interrupt exception, which provides a mechanism for applications to call operating system routines.

Syntax: SWI{<cond>} SWI_number

SWI	software interrupt	<i>lr_svc</i> = address of instruction following the SWI <i>spsr_svc</i> = <i>cpsr</i> <i>pc</i> = vectors + 0x8 <i>cpsr</i> mode = SVC <i>cpsr</i> I = 1 (mask IRQ interrupts)
-----	--------------------	---

- When the processor executes an SWI instruction, it sets the program counter *pc* to the offset 0x8 in the vector table.
- The instruction also forces the processor mode to SVC, which allows an operating system routine to be called in a privileged mode.
- The processor switches to ARM state and IRQ interrupts are disabled
- Each SWI instruction has an associated SWI number, which is used to represent a particular function call or feature.

Example

A simple example of an SWI call with SWI number 0x123456, used by ARM toolkits as a debugging SWI. Typically the SWI instruction is executed in user mode.

PRE

```

cpsr = nzcVqift_USER
pc = 0x00008000
lr = 0x003fffff; lr = r14
r0 = 0x12

```

```

0x00008000  SWI  0x123456

```

POST

```

cpsr = nzcVqift_SVC
spsr = nzcVqift_USER
pc = 0x00000008
lr = 0x00008004
r0 = 0x12

```

- Code called the *SWI handler* is required to process the SWI call.
- The handler obtains the SWI number using the address of the executed instruction, which is calculated from the link register *lr*.
- The SWI number is determined by

SWI_Number = <SWI instruction> **AND NOT** (0xff000000)

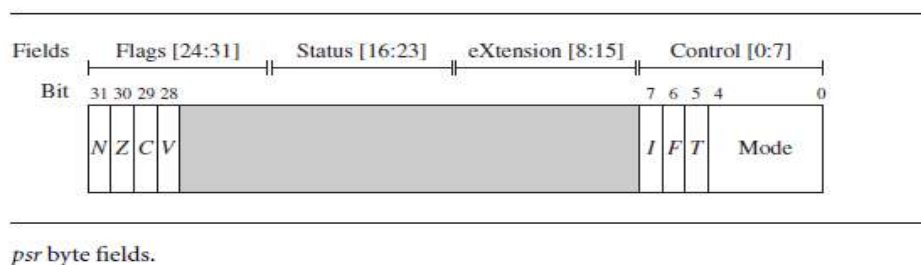
- Here the *SWI instruction* is the actual 32-bit SWI instruction executed by the processor.

PROGRAM STATUS REGISTER INSTRUCTIONS

The ARM instruction set provides two instructions to directly control a program status register (*psr*).

- The MRS instruction transfers the contents of either the *cpsr* or *spsr* into a register; in the reverse direction.
- The MSR instruction transfers the contents of a register into the *cpsr* or *spsr*.

Together these instructions are used to read and write the *cpsr* and *spsr*



- *Fields* can be any combination of control(*c*), extension (*x*), status (*s*), and flags (*f*).
- These fields relate to particular byte regions in a *psr*
- The *c* field controls the interrupt masks, Thumb state, and processor mode.

MRS	copy program status register to a general-purpose register	$Rd = psr$
MSR	move a general-purpose register to a program status register	$psr[field] = Rm$
MSR	move an immediate value to a program status register	$psr[field] = immediate$

MRS

Move to ARM register from status register (*cpsr* or *spsr*)

1. MRS<cond> Rd, cpsr
2. MRS<cond> Rd, spsr

These instructions set $Rd = cpsr$ and $Rd = spsr$, respectively. Rd must not be *pc*.

MSR

Move to status register (*cpsr* or *spsr*) from an ARM register

MSR<cond> cpsr_<fields>, #<rotated_immed>

$cpsr = (cpsr \& \sim\langle mask \rangle) | (\langle rotated_immed \rangle \& \langle mask \rangle)$

Example: Enabling IRQ interrupts by clearing the *I* mask.

The MSR first copies the *cpsr* into register *r1*.

The BIC instruction clears bit 7 of *r1*.

Register *r1* is then copied back into the *cpsr*, which enables IRQ interrupts.

PRE

cpsr = nzcqvIFt_SVC

MRS r1, cpsr

BIC r1, r1, #0x80 ; 0b01000000

MSR cpsr_c, r1

POST

cpsr = nzcqvIFt_SVC

This example is in *SVC* mode. In *user* mode reading of all *cpsr* bits can be done , but can only update the condition flag field *f*.

COPROCESSOR INSTRUCTIONS

- Coprocessor instructions are used to extend the instruction set.
- A coprocessor can either provide additional computation capability or be used to control the memory subsystem including caches and memory management.
- The coprocessor instructions include data processing, register transfer, and memory transfer instructions.

Syntax: CDP{<cond>} cp, opcode1, Cd, Cn [, opcode2]
 <MRC|MCR>{<cond>} cp, opcode1, Rd, Cn, Cm [, opcode2]
 <LDC|STC>{<cond>} cp, Cd, addressing

CDP	coprocessor data processing—perform an operation in a coprocessor
MRC MCR	coprocessor register transfer—move data to/from coprocessor registers
LDC STC	coprocessor memory transfer—load and store blocks of memory to/from a coprocessor

The *cp* field represents the coprocessor number between *p0* and *p15*.
 The *opcode* fields describe the operation to take place on the coprocessor.
 The *Cn*, *Cm*, and *Cd* fields describe registers within the coprocessor.
 The coprocessor operations and registers depend on the specific coprocessor

COPROCESSOR 15

- Coprocessor 15 (CP15) is reserved for system control purposes, such as memory management, write buffer control, cache control, and identification registers.
- CP15 configures the processor core and has a set of dedicated registers to store configuration Information.
- A value written into a register sets a configuration attribute—for example, switching on the cache.
- CP15 is called the *system control coprocessor*.
 - Both MRC and MCR instructions are used to read and write to CP15, where register *Rd* is the core destination register, *Cn* is the primary register, *Cm* is the secondary register, and *opcode2* is a secondary register modifier.

CONDITIONAL EXECUTION

- Conditional execution reduces the number of branches, which also reduces the number of pipeline flushes and thus improves the performance of the executed code.
- Conditional execution depends upon two components: the condition field and condition flags.
- The condition field is a two-letter mnemonic appended to the instruction mnemonic.
- The default mnemonic is AL, or *always execute*.
- The condition field is located in the instruction, and the condition flags are located in the *cpsr*.

Example: 1

An ADD instruction with the EQ condition appended. This instruction will only be executed when the zero flag in the *cpsr* is set to 1.

; r0 = r1 + r2 if zero flag is set

ADDEQ r0, r1, r2

Example: 2

Simple C code:

```
while (a!=b)
{
  if (a>b) a -= b; else b -= a;
}
```

Code written in ARM assembler:

This example code uses conditional execution on the branch instructions

Greatest Common Divisor Algorithm

gcd

```
CMP r1, r2
BEQ complete
BLT lessthan
SUB r1, r1, r2
B gcd
```

```
lessthan
SUB r2, r2, r1
B gcd
complete
```

Code with full conditional execution:

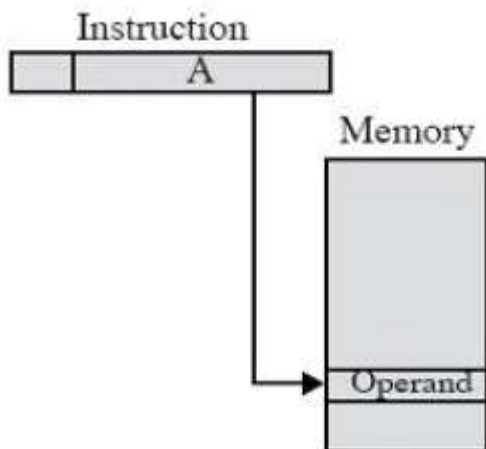
The code written using full condition execution reduces the number of instructions

```
gcd
CMP r1, r2
SUBGT r1, r1, r2
SUBLT r2, r2, r1
BNE gcd
```

Direct and Indirect Addressing

Direct Addressing

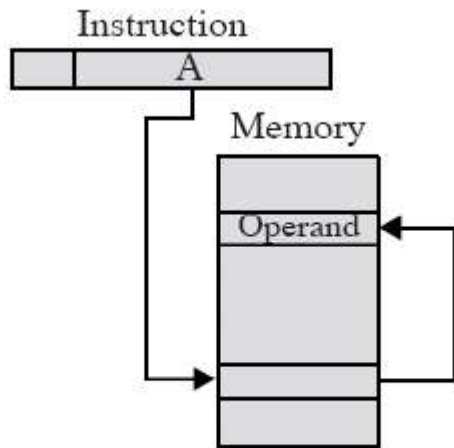
- Address field contains address of operand
- Effective address (EA) = address field (A) • e.g. add ax, count or add ax,[10FC]
- Look in memory at address for operand
- Single memory reference to access data
- No additional calculations to work out effective address



Memory-Indirect Addressing

- Memory cell pointed to by address field contains the address of (pointer to) the operand
- $EA = (A)$

— Look in A, find address (A) and look there for operand



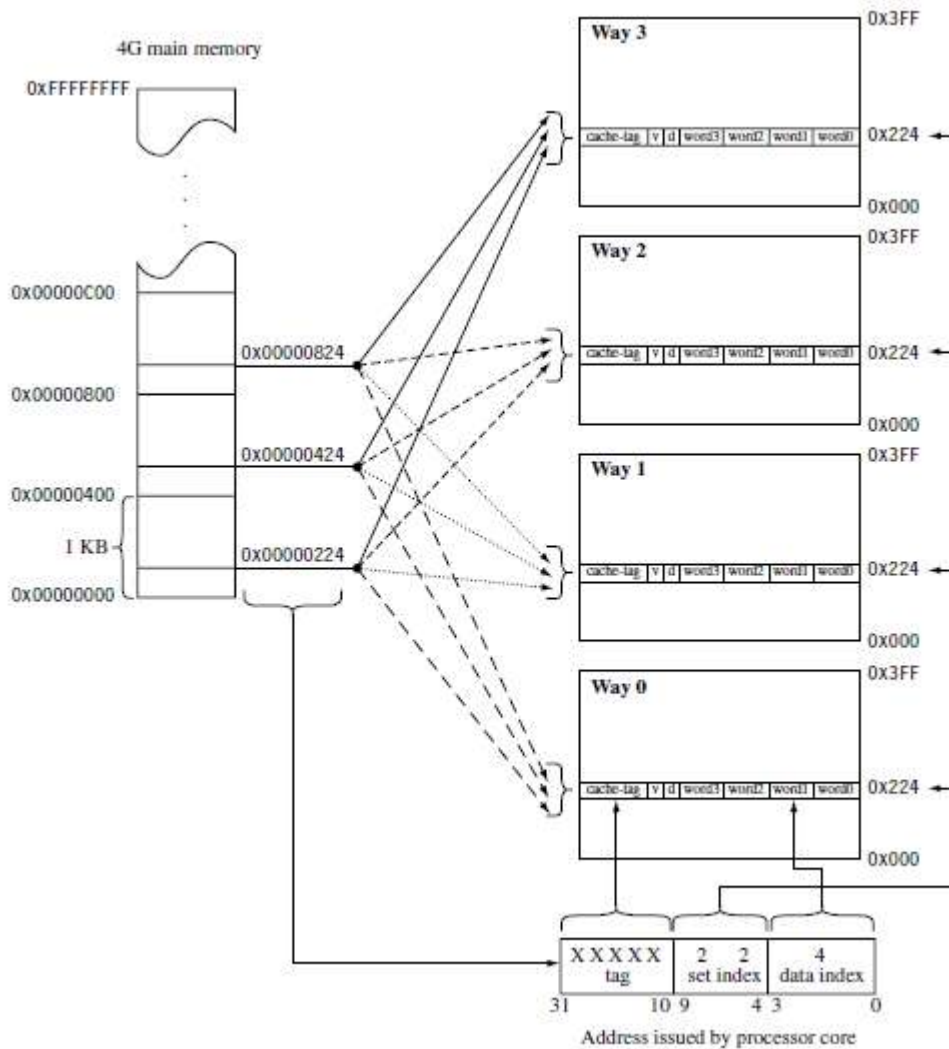
Associative cache:

A set-associative scheme is a hybrid between a fully associative cache, and direct mapped cache. It's considered a reasonable compromise between the complex hardware needed for fully associative caches (which requires parallel searches of all slots), and the simplistic direct-mapped scheme, which may cause collisions of addresses to the same slot (similar to collisions in a hash table).

Set Associativity

Some caches include an additional design feature to reduce the frequency of thrashing. This structural design feature is a change that divides the cache memory into smaller equal units, called ways. In a four KB cache; however, the set index now addresses more than one cache line—it points to one cache line in each way. Instead of one way of 256 lines, the cache has four ways of 64 lines. The four cache lines with the same set index are said to be in the same *set*, which is the origin of the name “set index.”

The set of cache lines pointed to by the set index are *set associative*. A data or code block from main memory can be allocated to any of the four ways in a set without affecting program behavior; in other words the storing of data in cache lines within a set does not affect program execution. Two sequential blocks from main memory can be stored as cache lines in the same way or two different ways. The important thing to note is that the data or code blocks from a specific location in main memory can be stored in any cache line that is a member of a set. The placement of values within a set is exclusive to prevent the same code or data block from simultaneously occupying two cache lines in a set. The mapping of main memory to a cache changes in a four-way set associative cache. The size of the area of main memory that maps to cache is now 1 KB instead of 4 KB. This means that the likelihood of mapping cache line data blocks to the same set is now four times higher. This is offset by the fact that a cache line is one fourth less likely to be evicted.



Increasing Set Associativity

As the associativity of a cache controller goes up, the probability of thrashing goes down. The ideal goal would be to maximize the set associativity of a cache by designing it so any main memory location maps to any cache line. A cache that does this is known as a *fully associative* cache. However, as the associativity increases, so does the complexity of the hardware that supports it. One method used by hardware designers to increase the set associativity of a cache includes a *content addressable memory* (CAM).

2 MARKS

1. What are data processing instructions? (Nov 2017)

The data processing instructions manipulate data within registers. They are move instructions, arithmetic instructions, logical instructions, comparison instructions, and multiply instructions. Most data processing instructions can process one of their operands using the barrel shifter.

2. What is barrel shifter?

A **barrel shifter** is a digital circuit that can shift a data word by a specified number of bits in one clock cycle. It can be implemented as a sequence of multiplexers (mux.)

3. State the unique feature of ARM in terms of instruction processing.

A unique and powerful feature of the ARM processor is the ability to shift the 32-bit binary pattern in one of the source registers left or right by a specific number of positions before it enters the ALU. This shift increases the power and flexibility of many data processing operations.

4. What is the use of barrel shifter?

Pre-processing or shift occurs in the barrel shifter within the cycle time of the instruction. This is particularly useful for loading constants into a register and achieving fast multiplies or division by a power of 2.

5. What is RSB? Give example.

Reverse subtract instruction (RSB) subtracts $r1$ from the constant value #0, writing the result to $r0$. You can use this instruction to negate numbers.

PRE

$r0 = 0x00000000$

$r1 = 0x00000077$

RSB $r0, r1, \#0$; $Rd = 0x0 - r1$

POST

$r0 = -r1 = 0xfffff89$

6. Give an example to illustrate use of inline barrel shifter

Register $r1$ is first shifted one location to the left to give the value of twice $r1$. The ADD instruction then adds the result of the barrel shift operation to register $r1$. The final result transferred into register $r0$ is equal to three times the value stored in register $r1$.

PRE

$r0 = 0x00000000$

$r1 = 0x00000005$

ADD $r0, r1, r1, LSL \#1$

POST

$r0 = 0x0000000f$

$r1 = 0x00000005$

7. List out the logical instructions of ARM processor.

AND	logical bitwise AND of two 32-bit values	$Rd = Rn \& N$
ORR	logical bitwise OR of two 32-bit values	$Rd = Rn N$
EOR	logical exclusive OR of two 32-bit values	$Rd = Rn \wedge N$
BIC	logical bit clear (AND NOT)	$Rd = Rn \& \sim N$

8. When do the CPSR flags updated?

The arithmetic and logical instructions update the *cpsr* flags only if the S suffix is present in the instruction. But when using comparison instructions, the *cpsr* flag bits are updated automatically according to the result of the operation.

9. Give example for LSL operation.

PRE

```
cpsr = nzcqvqiFt_USER  
r0 = 0x00000000  
r1 = 0x80000004  
MOVS r0, r1, LSL #1
```

POST

```
cpsr = nzCvqiFt_USER  
r0 = 0x00000008  
r1 = 0x80000004
```

MOVS instruction shifts register *r1* left by one bit. This multiplies register *r1* by a value 2^1 . The *C* flag is updated in the *cpsr* because the S suffix is present in the instruction mnemonic.

10. Is compare instruction a subtraction instruction?

The CMP is effectively a subtract instruction with the result discarded. The results are discarded but the condition bits are updated in the *cpsr* comparison instructions only modify the condition flags of the *cpsr* and do not affect the registers being compared.

11. Where does the result of long multiply instruction stored?

The long multiply instructions (SMLAL, SMULL, UMLAL, and UMULL) produce a 64-bit result. The result is too large to fit a single 32-bit register so the result is placed in two registers labeled *RdLo* and *RdHi*. *RdLo* holds the lower 32 bits of the 64-bit result, and *RdHi* holds the higher 32 bits of the 64-bit result.

12. What are branch instructions?

A branch instruction changes the flow of execution or is used to call a routine. This type of instruction allows programs to have subroutines, *if-then-else* structures, and loops. The change of execution flow forces the program counter *pc* to point to a new address.

13. What are types of load-store instructions?/ state the use of Load –store instructions in ARM (Nov 2017)

Load-store instructions transfer data between memory and processor registers. There are three types of load-store instructions: single-register transfer, multiple-register transfer, and swap.

14. Explain about pre-indexing with write back.

Preindex with writeback calculates an address from a base register plus address offset and then updates that address base register with the new address.

PRE

```
r0 = 0x00000000  
r1 = 0x00090000  
mem32[0x00009000] = 0x01010101  
mem32[0x00009004] = 0x02020202  
LDR r0, [r1, #4]!
```

Preindexing with writeback:

POST(1)

r0 = 0x02020202

r1 = 0x00009004

15. State the advantage of multiple register transfer over single register transfers.

Multiple-register transfer instructions are more efficient from single-register transfers for moving blocks of data around memory and saving and restoring context and stacks. Load-store multiple instructions can increase interrupt latency.

16. State the operations of BNE instruction

The BNE is the branch instruction B with a condition mnemonic NE (not equal). If the previous compare instruction sets the condition flags to *not equal*, the branch instruction is executed.

17. Define push and pop operations of stack.

The ARM architecture uses the load-store multiple instructions to carry out stack operations. The *pop* operation (removing data from a stack) uses a load multiple instruction; similarly, the *push* operation (placing data onto the stack) uses a store multiple instruction.

18. Define ascending and descending stack.

A stack is either *ascending* (A) or *descending* (D). Ascending stacks grow towards higher memory addresses; in contrast, descending stacks grow towards lower memory addresses.

19. What is a full stack?

In a *full stack* (F), the stack pointer *sp* points to an address that is the last used or full location (i.e., *sp* points to the last item on the stack).

20. State the position of *sp* in an empty stack.

In an *empty stack* (E) the *sp* points to an address that is the first unused or empty location (i.e., it points after the last item on the stack).

21. What are swap instructions?

The swap instruction is a special case of a load-store instruction. It swaps the contents of memory with the contents of a register. This instruction is an *atomic operation*—it reads and writes a location in the same bus operation, preventing any other instruction from reading or writing to that location until it completes.

22. When can swap instruction be implemented?

The swap instruction loads a word from memory into register *r0* and overwrites the memory with register *r1*. This instruction is particularly useful when implementing semaphores and mutual exclusion in an operating system.

23. State the instructions required to control the PSR./ List out the PSR instructions (Nov 2016) (May 2017)

The ARM instruction set provides two instructions to directly control a program status register (*psr*). The MRS instruction transfers the contents of either the *cpsr* or *spsr* into a register; in the reverse direction, the MSR instruction transfers the contents of a register into the *cpsr* or *spsr*.

24. What is a co-processor?

A coprocessor can either provide additional computation capability or be used to control the memory subsystem including caches and memory management.

25. What are co-processor instructions?

Coprocessor instructions are used to extend the instruction set. The coprocessor instructions include data processing, register transfer, and memory transfer instructions.

26. What is CP15?

Coprocessor 15 (CP15) is reserved for system control purposes, such as memory management, write buffer control, cache control, and identification registers. CP15 configures the processor core and has a set of dedicated registers to store configuration information. CP15 is called the *system control coprocessor*.

27. What is conditional execution? State its advantage.

Most ARM instructions are *conditionally executed*— can specify that the instruction only executes if the condition code flags pass a given condition or test. By using conditional execution instructions you can increase performance and code density. Conditional execution reduces the number of branches, which also reduces the number of pipeline flushes and thus improves the performance of the executed code.

28. What are the fields necessary to perform conditional execution?

Conditional execution depends upon two components: the condition field and condition flags. The condition field is located in the instruction, and the condition flags are located in the *cpsr*.

Eg:

$r0 = r1 + r2$ if zero flag is set

ADDEQ r0, r1, r2

This instruction will only be executed when the zero flag in the *cpsr* is set to 1.

29. Define ATPCS

ARM has specified an ARM-Thumb Procedure Call Standard (ATPCS) that defines how routines are called and how registers are allocated. In the ATPCS, stacks are defined as being full descending stacks.

30. Define stack base, stack pointer and stack limit

When handling a checked stack there are three attributes that need to be preserved: the *stack base*, the *stack pointer*, and the *stack limit*. The stack base is the starting address of the stack in memory. The stack pointer initially points to the stack base; as data is pushed on to the stack, the stack pointer descends memory and continuously points to the top of stack. If the stack pointer passes the stack limit, then a *stack overflow error* has occurred.

31. State the purpose of NOP

NOP - No operation

This is not an ARM instruction. It is an assembly macro that produces an instruction having no effect other than advancing the *pc* as normal.

32. What is the use of BKPT?

BKPT - Breakpoint instruction

It is used to set breakpoints in a program

Syntax:

BKPT <immed16>

33. State the use of BXJ

BXJ is a Branch with exchange instruction

If the JE (Java Enable) configuration bit is clear, then BXJ behaves as a BX. Otherwise, the behavior is defined by the architecture of the Java Extension hardware. It sets $J = 1$ in the *cpsr* and starts executing Java instructions from a general purpose register designated as the Java program counter *jpc*.

34. Give the syntax of REV instruction with example

REV - Reverse bytes within a word or halfword.

Syntax:

REV<cond> Rd, Rm

Rd[31:24] = Rm[07:00]; Rd[23:16] = Rm[15:08];

Rd[15:08] = Rm[23:16]; Rd[07:00] = Rm[31:24];

Example:

REV r0, r0; switch endianness of a word

35. Implement the statement $x=(a+b)-c$, using ARM instructions. (May 2016)

ADD r0,r0,r1; // r0=r0+r1

SUB r3,r0-r2

36. What is instruction set? (Nov 2016)

An **instruction set**, or **instruction set architecture (ISA)**, is the part of the computer architecture related to programming, including the native data types, instructions, registers, addressing modes, memory architecture, interrupt and exception handling, and external I/O. An ISA includes a specification of the set of opcodes (machine language), and the native commands implemented by a particular processor.

Instruction set is a complete set of all the instructions in machine code that can be recognized and executed by a central processing unit.